BACHELOR'S THESIS (TFG) · UC3M

Warm start experiment

A step-by-step guide to testing Shrink & Perturb on CARLA (you write the code)

[Your name] · Supervisor: [name] · September 2026

Goal

Every uploaded flight means a retrain. This experiment answers one question: **when new data arrives, where should the next training round start from?** It tests the claim of Ash & Adams (NeurIPS 2020, `papers/warm_start.pdf`) that continuing from last round's weights hurts generalization, and that *Shrink & Perturb* fixes it, on YOLO and drone imagery instead of ResNets and CIFAR.

This guide gives the steps, the reasons and the questions to answer. It deliberately does not give the finished code.

Step 0 · Understand the claim

Read sections 1–3 of the paper and Figure 1, then section 4 ("Shrink, Perturb, Repeat").

Write down, in your own words:

1. Why can a warm-started model have the same training loss as a fresh one, but a worse test score?
2. What does the paper say about the gradients from old vs. new data (Figure 5)?
3. Why didn't changing the learning rate, batch size or regularization fix it (section 3)?

If you can answer these three without the paper in front of you, you can defend this chapter.

Step 1 · Simulate flights arriving

You don't need new flights. Split the labelled data you already have into **3–5 rounds** that arrive one at a time.

- Split **by flight or by date**, never randomly by tile or frame.
- Keep a **fixed test set** that no round ever trains on, and a separate **validation set** (YOLO uses it to choose `best.pt`).
- Each round **k** trains on everything from rounds 1…**k** (the paper's "data arrives piecemeal" setting).

Question: why would a random split hide the warm-start effect? (Hint: how different would round 2 be from round 1?)

Step 2 · The three setups

Setup	Starting weights for round k
Cold	COCO-pretrained weights, every round
Warm	The final weights of round **k−1**
Shrink & Perturb	$\lambda \cdot \theta_{k-1} + \text{noise}$

Caveat: The paper's "cold" start uses **random** weights. Yours uses **COCO-pretrained** ones, since nobody trains YOLO from scratch on a few hundred images. This difference belongs in your thesis.

Step 3 · Implement Shrink & Perturb

The paper's update (section 4) is $\theta_t \leftarrow \lambda\theta_{t-1} + p_t$ with $0 < \lambda < 1$. In practice the noise p_t is **a freshly initialized network multiplied by a small "noise scale"**. The paper's main setting is $\lambda = 0.6$ with noise scale 0.01 (Figure 7).

What you need to build (roughly 10–15 lines):

1. Load last round's model and a second "noise" model.
2. Walk through both `state_dict()`s together and mix each tensor.
3. Make sure the training run actually starts from the mixed weights.

Ultralytics hints (API details only):

- `YOLO("yolo11n.yaml")` builds a **randomly initialized** model; `YOLO("yolo11n.pt")` loads **COCO** weights.
- The PyTorch network is in `model.model`, so its weights are `model.model.state_dict()`.
- Training writes `weights/last.pt` and `weights/best.pt`. Decide which one the next round uses, and use the same choice for every setup.

Decide these yourself. Each one is a possible tribunal question:

- **What is the noise?** A random-init network (as in the paper) or COCO weights? Is "shrinking toward COCO" a different idea?
- **Which tensors do you skip?** BatchNorm `running_mean`, `running_var` and `num_batches_tracked` are statistics, not learned weights.
- **How do you check it worked?** Print one weight before mixing, after mixing, and at the start of training.
- Ultralytics checkpoints store **EMA** weights. Which weights are you actually shrinking?

Step 4 · Keep everything else identical

- The same number of epochs per round for all setups. Otherwise you're comparing training budgets, not starting points.
- YOLO resets the optimizer every run and uses warmup, a learning-rate schedule and heavy augmentation. The paper had none of these, so the effect may be smaller. **That is still a valid result.**
- Fixed seeds (`seed=`, `deterministic=True`) and at least 3 seeds per setup.

Step 5 · What to measure

Measurement	Why
Test mAP50 after each round	The main figure: one line per setup, shaded band over seeds
Training time per round	Warm-starting is only worth it if it saves time
Train loss vs. test mAP	Do you see the paper's "same training loss, worse test score"?
Optional: $\lambda \in \{0.3, 0.6, 0.9\}$	Reproduces the shape of Figure 8 on your data

Step 6 · Start small

1. YOLO-nano, 2 rounds, 3–5 epochs, 1 seed. Check that all three setups run end to end.
2. Time one epoch. Multiply by setups × rounds × seeds × epochs to estimate the full run.
3. Then scale up to the real setting.

Hardware

A GPU is needed for the full experiment; a CPU is enough for Step 6.1.

Running it on Google Colab

The free tier usually gives a **T4 GPU (16 GB)**, which is plenty for YOLO-nano. The limits are the sessions, not the GPU:

- Sessions disconnect when idle and have a maximum length.
- A GPU is not always available.
- The local disk is wiped when the session ends.

So plan for the full run to be split across several sessions.

Setup

1. `Runtime → Change runtime type → T4 GPU`. Check it with `!nvidia-smi`.
2. Install Ultralytics: `!pip install ultralytics`. Write down the version; use the same one in every session.
3. Mount Drive: `from google.colab import drive; drive.mount("/content/drive")`.
4. Keep your **code, round splits and results** on Drive.
5. At the start of each session, **copy the dataset** (ideally one `.zip`) from Drive to `/content/` and unzip it there. Training straight from Drive is very slow.

Surviving disconnects

Treat each (setup, round, seed) as one **job** with its own output folder on Drive, for example `results/sp_l0.6/round2/seed1/`.

- At the end of a job, write a small "done" file with its metrics.
- Before starting a job, check for that file and skip the job if it exists.
- Then rerunning the notebook after a disconnect just continues where it stopped.

Question: round **k** of a warm or Shrink & Perturb run needs round **k−1**'s weights. Where must those be saved so that a new session can find them?

Things to watch

- Set `project=` and `name=` in `model.train()` so outputs go to Drive, or copy them there right after each job.
- Keep `batch`, `imgsz` and `workers` the same in every session. Colab machines can differ, and your timings should be comparable.
- Training time on Colab is noisy, since you share the hardware. For the "training time" measurement, also report epochs, or run the timed jobs in one session.
- Do the debugging (Step 6.1) on your laptop's CPU, and save Colab GPU time for the real runs.

My proposal (not from the papers): After it works, the same three-way comparison becomes "option D" in the plan: a small bandit that chooses between continue, Shrink & Perturb and restart after each flight.